

Contents

- [PREAMBLE](#)
- =====
- [1. COMMENTING](#)
- =====
- =====
- [2. DFT and DTFT](#)
- =====
- =====
- [3. DFT and IDFT](#)
- =====
- [4. DFT AND FFT](#)
- =====
- [Functions provided for the lab](#)

PREAMBLE

DO NOT REMOVE THE LINE BELOW

```
clear; close all;
```

=====

1. COMMENTING

=====

MAKE SURE FILE BELOW IS IN SAME DIRECTORY AS THIS FILE

```
type('eel3135_lab10_comment.m')
```

```
%% Question 1 Comment Code  
clear all; clc; close all;
```

```
fs = 16000;  
tt = 0:1/fs:0.01-1/fs;
```

```
w0 = 888*pi;  
w1 = 3520*pi;  
w2 = 14080*pi;
```

```
x = 0.5 + cos(w0*tt) + cos(w1*tt + pi/4) + cos(w2*tt + 2*pi/3);
```

```
N = [50,length(x),500,1000];
```

```
X1 = fft(x,N(1));  
X2 = fft(x,N(2));  
X3 = fft(x,N(3));
```

```

X4 = fft(x,N(4));

% Answer in your comments: How is the DFT (using the FFT algorithm)
% calculated when N is larger than the length of X?
% (Hint: read help FFT)
%The DFT is calculated when N is larger than the length of X by adding
%zeros to the end so that they become the same length.

% Answer on your comments: How is the DFT (using the FFT algorithm)
% calculated when N is smaller than the length of X?
%The DFT is calculated when N is smaller than the length of X by only
%considering the first N samples and not considering the rest of the data.

% Answer in your comments: Why would you not want to always make N as large
% as possible?
%Because it would not provide any useful information and would require more
%computational power. Making N as large as possible in MATLAB would have
%the simulation run for a very long time for no reason when you can get
% the same result by having a more specific value of N.

b1 = -ceil((N(1)-1)/2):floor((N(1)-1)/2);
b2 = -ceil((N(2)-1)/2):floor((N(2)-1)/2);
b3 = -ceil((N(3)-1)/2):floor((N(3)-1)/2);
b4 = -ceil((N(4)-1)/2):floor((N(4)-1)/2);

% Answer in your comments: What frequencies do the peaks represent?
% The peaks represent the fundamental frequencies from w0, w1 and w2.

f1 = b1*fs/N(1);
f2 = b2*fs/N(2);
f3 = b3*fs/N(3);
f4 = b4*fs/N(4);

w1 = 2*pi/N(1)*b1;
w2 = 2*pi/N(2)*b2;
w3 = 2*pi/N(3)*b3;
w4 = 2*pi/N(4)*b4;

figure
subplot(221)
plot(b1,fftshift(abs(X1)))
hold on;
plot(b1,fftshift(abs(X1)), '.', 'markersize', 8)
hold off;
xlim([-N(1)/2 N(1)/2]); ylim([0 100]);
xlabel('Bin Number')
ylabel('Magnitude')
title(sprintf('Length %i DFT',N(1)))

subplot(222)
plot(b2,fftshift(abs(X2)))
hold on;
plot(b2,fftshift(abs(X2)), '.', 'markersize', 8)
hold off;
xlim([-N(2)/2 N(2)/2]); ylim([0 100]);
xlabel('Bin Number')
ylabel('Magnitude')
title(sprintf('Length %i DFT',N(2)))

```

```

subplot(223)
plot(b3,fftshift(abs(X3)))
hold on;
plot(b3,fftshift(abs(X3)), '.', 'markersize', 8)
hold off;
xlim([-N(3)/2 N(3)/2]); ylim([0 100]);
xlabel('Bin Number')
ylabel('Magnitude')
title(sprintf('Length %i DFT',N(3)))

```

```

subplot(224)
plot(b4,fftshift(abs(X4)))
hold on;
plot(b4,fftshift(abs(X4)), '.', 'markersize', 8)
hold off;
xlim([-N(4)/2 N(4)/2]); ylim([0 81]);
xlabel('Bin Number')
ylabel('Magnitude')
title(sprintf('Length %i DFT',N(4)))

```

```

figure
subplot(221)
plot(f1,fftshift(abs(X1)))
hold on;
plot(f1,fftshift(abs(X1)), '.', 'markersize', 8)
hold off;
xlim([-fs/2 fs/2]); ylim([0 81]);
xlabel('Frequency (Hz)')
ylabel('Magnitude')
title(sprintf('Length %i DFT',N(1)))

```

```

subplot(222)
plot(f2,fftshift(abs(X2)))
hold on;
plot(f2,fftshift(abs(X2)), '.', 'markersize', 8)
hold off;
xlim([-fs/2 fs/2]); ylim([0 81]);
xlabel('Frequency (Hz)')
ylabel('Magnitude')
title(sprintf('Length %i DFT',N(2)))

```

```

subplot(223)
plot(f3,fftshift(abs(X3)))
hold on;
plot(f3,fftshift(abs(X3)), '.', 'markersize', 8)
hold off;
xlim([-fs/2 fs/2]); ylim([0 81]);
xlabel('Frequency (Hz)')
ylabel('Magnitude')
title(sprintf('Length %i DFT',N(3)))

```

```

subplot(224)
plot(f4,fftshift(abs(X4)))
hold on;
plot(f4,fftshift(abs(X4)), '.', 'markersize', 8)

```

```

hold off;
xlim([-fs/2 fs/2]); ylim([0 81]);
xlabel('Frequency (Hz)')
ylabel('Magnitude')
title(sprintf('Length %i DFT',N(4)))

```

```

figure
subplot(221)
plot(w1,fftshift(abs(X1)))
hold on;
plot(w1,fftshift(abs(X1)), '.', 'markersize', 8)
hold off;
xlim([-pi pi]); ylim([0 81]);
xlabel('Normalized Frequency (rad/s)');
ylabel('Magnitude')
title(sprintf('Length %i DFT',N(1)))

```

```

subplot(222)
plot(w2,fftshift(abs(X2)))
hold on;
plot(w2,fftshift(abs(X2)), '.', 'markersize', 8)
hold off;
xlim([-pi pi]); ylim([0 81]);
xlabel('Normalized Frequency (rad/s)');
ylabel('Magnitude')
title(sprintf('Length %i DFT',N(2)))

```

```

subplot(223)
plot(w3,fftshift(abs(X3)))
hold on;
plot(w3,fftshift(abs(X3)), '.', 'markersize', 8)
hold off;
xlim([-pi pi]); ylim([0 81]);
xlabel('Normalized Frequency (rad/s)');
ylabel('Magnitude')
title(sprintf('Length %i DFT',N(3)))

```

```

subplot(224)
plot(w4,fftshift(abs(X4)))
hold on;
plot(w4,fftshift(abs(X4)), '.', 'markersize', 8)
hold off;
xlim([-pi pi]); ylim([0 81]);
xlabel('Normalized Frequency (rad/s)')
ylabel('Magnitude')
title(sprintf('Length %i DFT',N(4)))

```

% Answer in your comments: How does the length of the DFT affect the
 % magnitude? Be sure to zoom in on the bases of the peaks in each DFT
 % magnitude plot.

% Increasing the lengths makes the peaks more defined and provides a better
 % and cleaner plot that has increased magnitude on the important

% frequencies.

2. DFT and DTFT

```
% CREATE SUM OF SINUSOIDS
n = 0:59;
x1 = 0.75 + cos(pi*n/20) + cos(pi*n/15) + cos(pi*n + 2*pi/3);

% USE THE FOLLOWING FOR THE DTFT
w_DTFT = linspace(0, 2*pi-pi/10000, 10000);

% NOTE: USE THE FOLLOWING COMMENTED LINE FOR PLOTTING THE DFT ATOP THE DTFT
% (YOU NEED TO DEFINE w_DFT), THIS WILL MAKE THE PLOTS EASIER TO INTERPRET
% plot(w_DTFT,abs(X_DTFT));
% hold on; plot(w_DFT,abs(X_DFT),'.', 'markersize', 10); hold off;

% -----
% 2(a)
% -----
% Only need to modify function -- this area can be empty

% -----
% 2(b)
% -----
X_DTFT = DTFT(x1, w_DTFT);

figure;
plot(w_DTFT, abs(X_DTFT));
xlabel('Normalized Freq (rad/sample)');
ylabel('Magnitude');
title('DTFT Magnitude of x');
xlim([0 2*pi]);
grid on;

% -----
% 2(c)
% -----
X_DFT = DFT(x1);
w_DFT = (2*pi/length(x1))*(0:length(x1)-1);

figure;
plot(w_DTFT, abs(X_DTFT));
hold on;
plot(w_DFT, abs(X_DFT), '.', 'markersize', 10);
hold off;
xlabel('Normalized Freq (rad/sample)');
ylabel('Magnitude');
title('60 point DFT of x');
xlim([0 2*pi]);
```

```

grid on;

% -----
% 2(d)
% -----

x1_55 = x1(1:55);
X_DFT_55 = DFT(x1_55);
w_DFT_55 = (2*pi/55)*(0:54);

figure;
plot(w_DTFT, abs(X_DTFT));
hold on;
plot(w_DFT_55, abs(X_DFT_55), '.', 'markersize', 10);
hold off;
xlabel('Normalized Freq (rad/sample)');
ylabel('Magnitude');
title('55 point DFT of x');
xlim([0 2*pi]);
grid on;

% -----
% 2(e)
% -----

x1_65 = [x1, zeros(1,5)];
X_DFT_65 = DFT(x1_65);
w_DFT_65 = (2*pi/65)*(0:64);

figure;
plot(w_DTFT, abs(X_DTFT));
hold on;
plot(w_DFT_65, abs(X_DFT_65), '.', 'markersize', 10);
hold off;
xlabel('Normalized Freq (rad/sample)');
ylabel('Magnitude');
title('65 point DFT of x');
xlim([0 2*pi]);
grid on;

% -----
% 2(f)
% -----

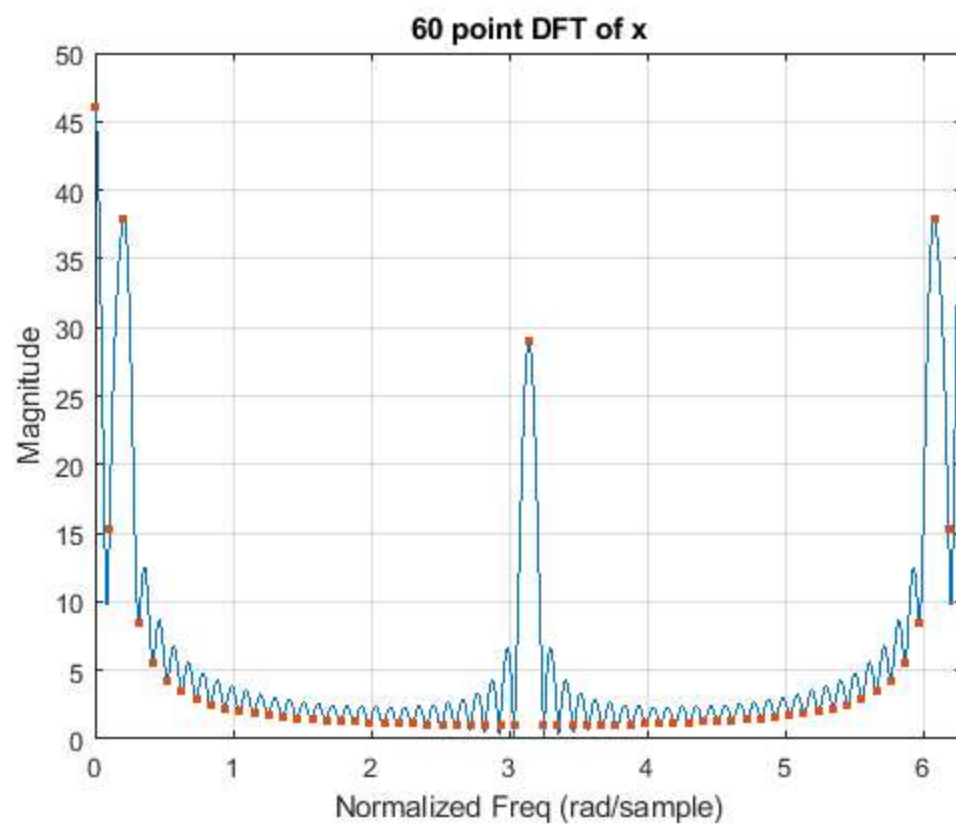
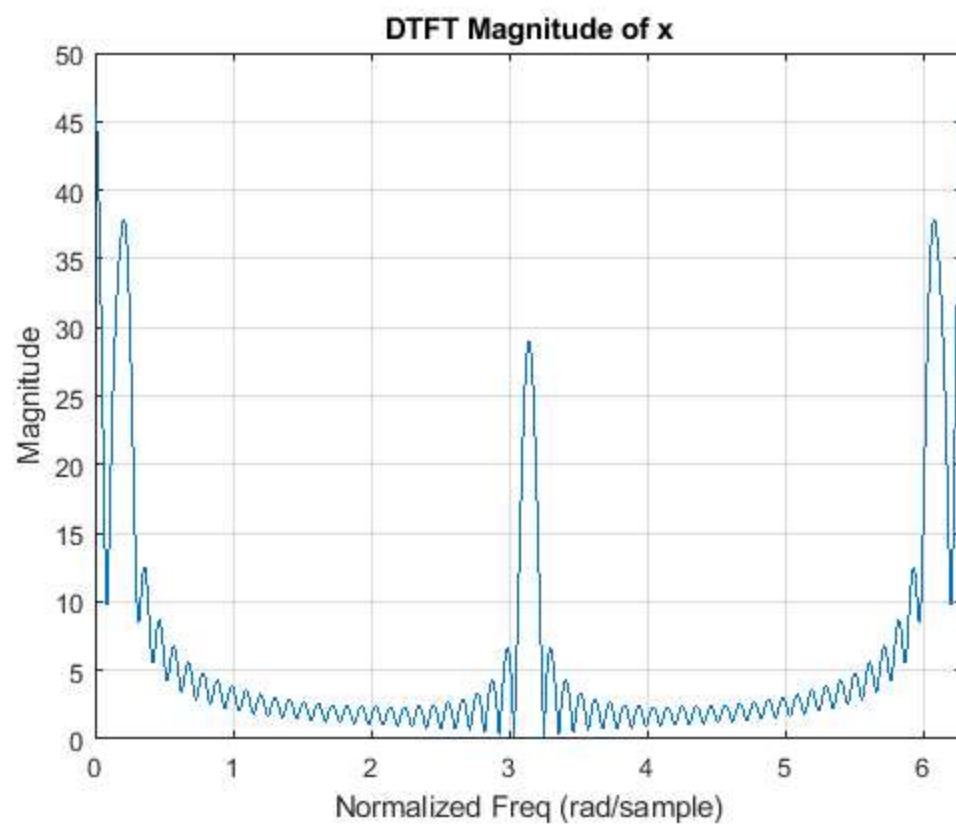
x1_200 = [x1, zeros(1, 200-length(x1))];
X_DFT_200 = DFT(x1_200);
w_DFT_200 = (2*pi/200)*(0:199);

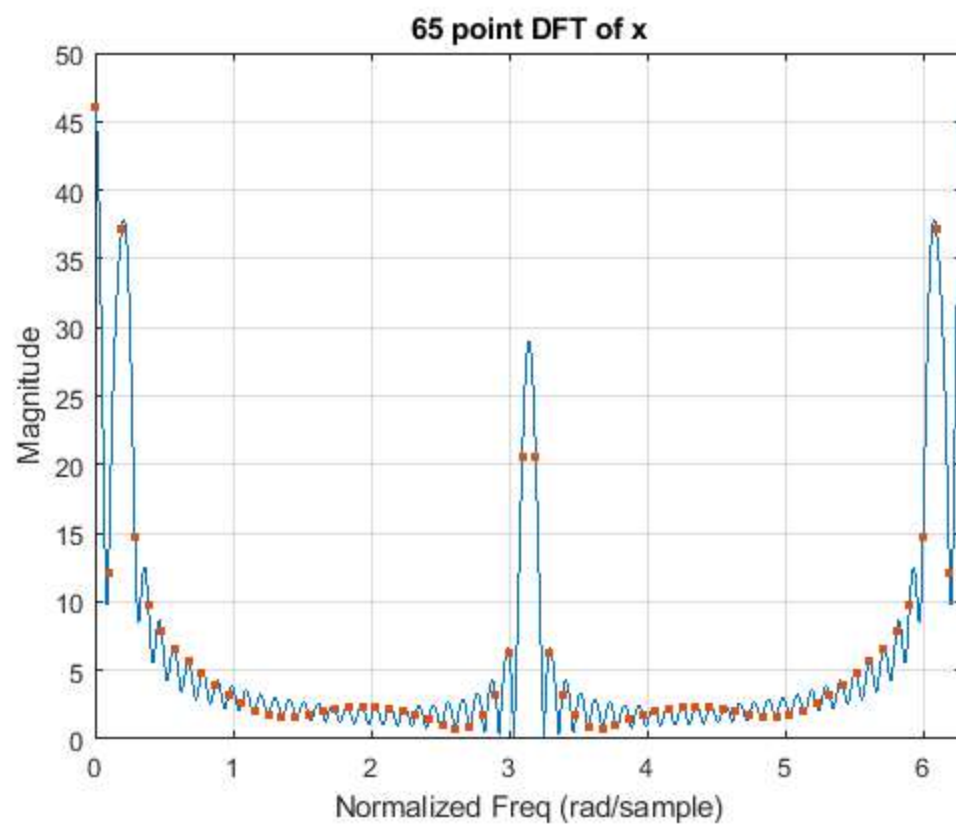
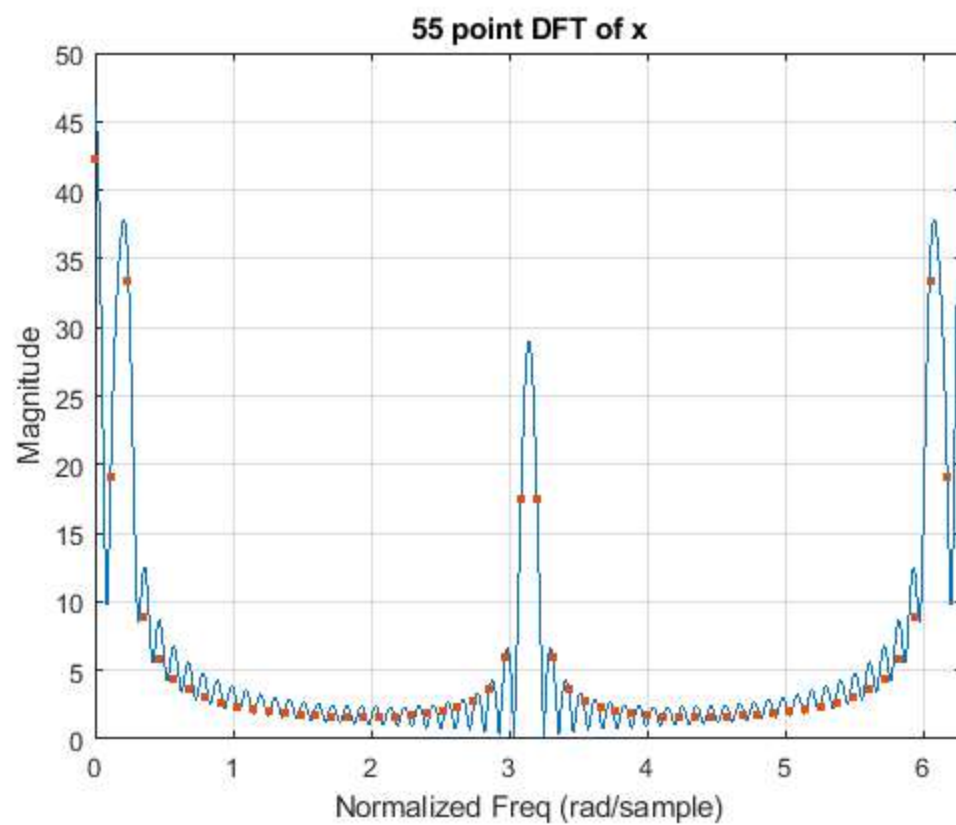
figure;
plot(w_DTFT, abs(X_DTFT));
hold on;
plot(w_DFT, abs(X_DFT), '.', 'markersize', 10);
plot(w_DFT_55, abs(X_DFT_55), '.', 'markersize', 10);
plot(w_DFT_65, abs(X_DFT_65), '.', 'markersize', 10);
plot(w_DFT_200, abs(X_DFT_200), '.', 'markersize', 10);
hold off;
xlabel('Normalized Freq (rad/sample)');
ylabel('Magnitude');

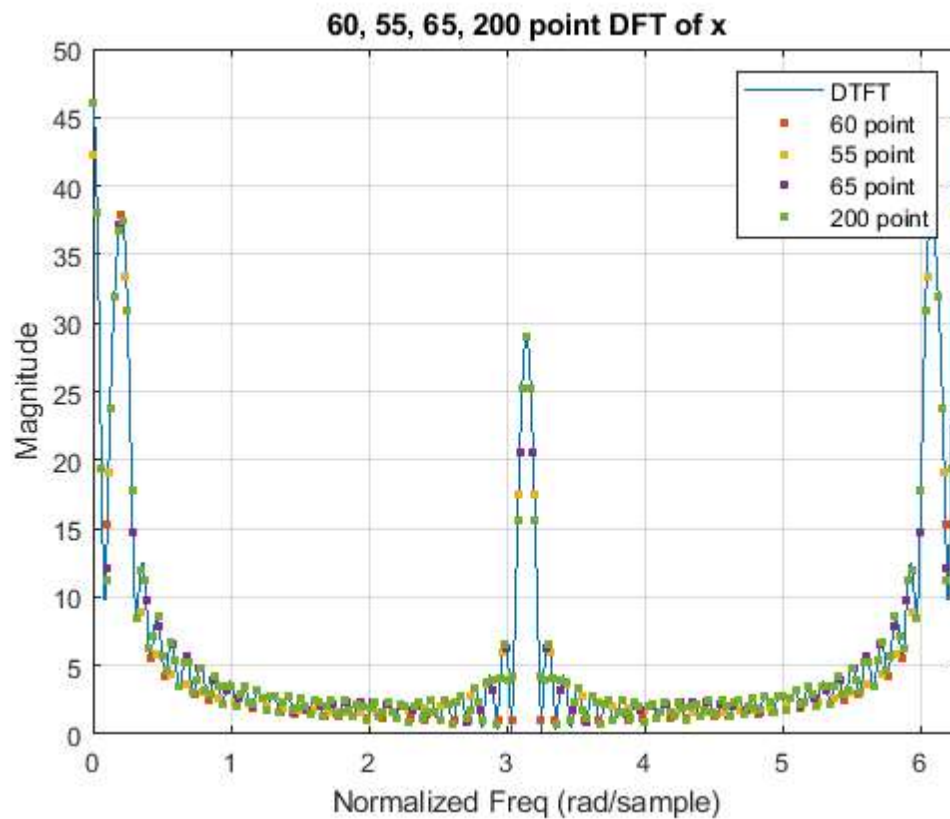
```

```
title('60, 55, 65, 200 point DFT of x');
xlim([0 2*pi]);
legend('DTFT', '60 point', '55 point', '65 point', '200 point');
grid on;

% -----
% 2(g)
% -----
% ANSWER QUESTION BELOW
% The relationship between the DTFT and the DFT is that the DFT are samples
% of the DTFT. As N increases, the DFT gets closer to the DTFT. They will
% have the same result when N is the exact same length as the signal
% and if the signal frequencies fall on the DFT frequencies by being
% integer multiples of  $2\pi/N$ 
```







3. DFT and IDFT

----- 3(a) ----- Only need to modify function --
 this area can be empty

```
% -----
% 3(b)
% -----
n2 = 0:99;
x2 = (n2 >= 30) - (n2 >= 65);
%length of x2 is approximately 65-30 = N = 35

figure;
stem(n2, x2);
xlabel('n (samples)');
ylabel('Amplitude');
title('u[n-30] - u[n-65]');
xlim([0 99]);
grid on;

% -----
% 3(c)
% -----
N_conv = 2*length(x2) - 1;

X2 = DFT([x2, zeros(1, N_conv - length(x2))]);
Y2 = X2 .* X2;
```

```

y2 = real(IDFT(Y2));

figure;
stem(0:length(y2)-1, y2);
xlabel('n');
ylabel('Amplitude');
title('Convolution');
xlim([0 99]);
grid on;

% -----
% 3(d)
% -----
N100 = 100;
x2_100 = [x2, zeros(1, N100 - length(x2))];
X2_100 = DFT(x2_100);
Y2_100 = X2_100 .* X2_100;
y2_100 = real(IDFT(Y2_100));

figure;
stem(0:99, y2_100);
xlabel('n');
ylabel('Amplitude');
title('IDFT N=100');
xlim([0 99]);
grid on;

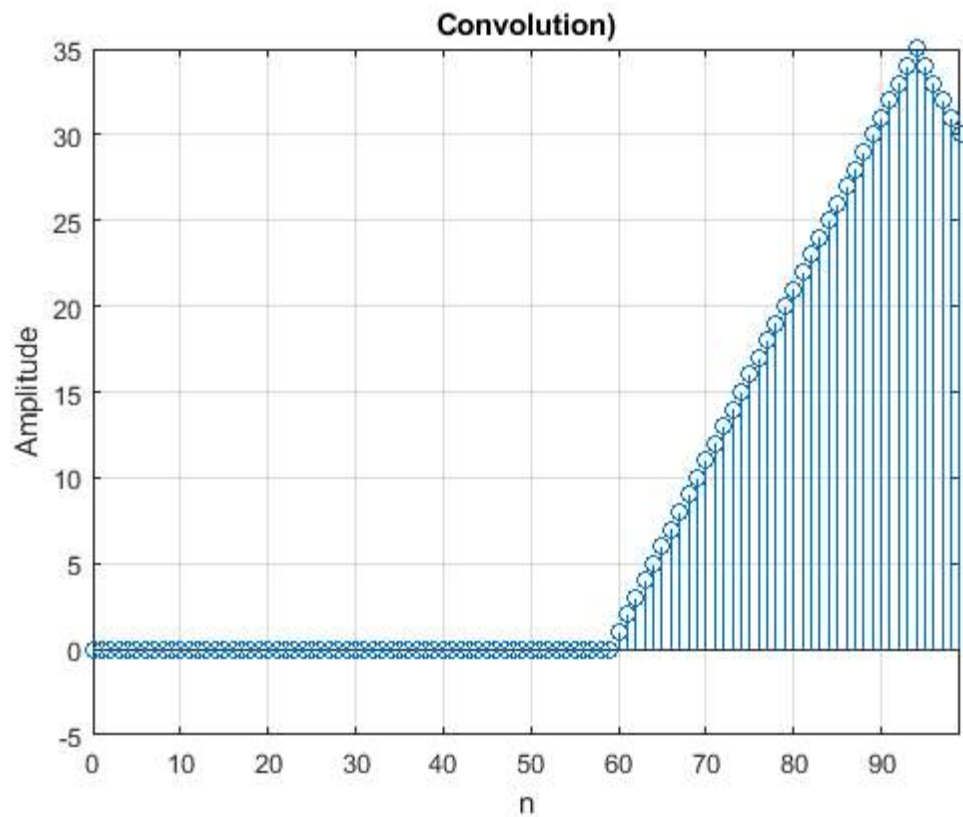
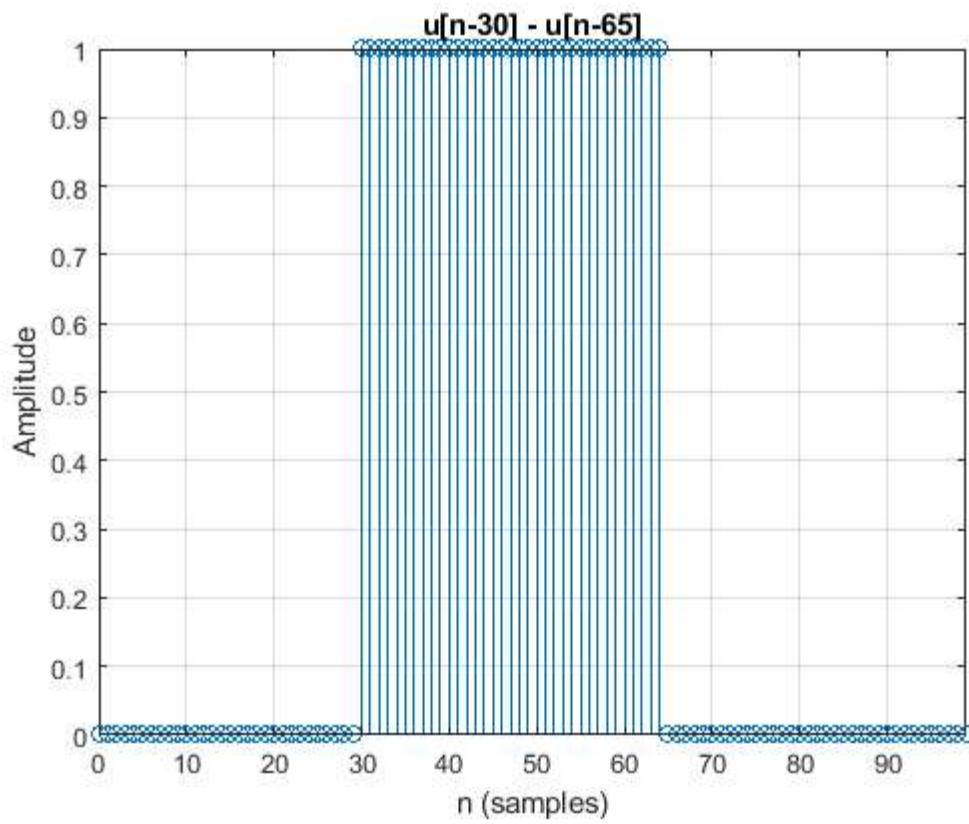
% -----
% 3(e)
% -----
N200 = 200;
x2_200 = [x2, zeros(1, N200 - length(x2))];
X2_200 = DFT(x2_200);
Y2_200 = X2_200 .* X2_200;
y2_200 = real(IDFT(Y2_200));

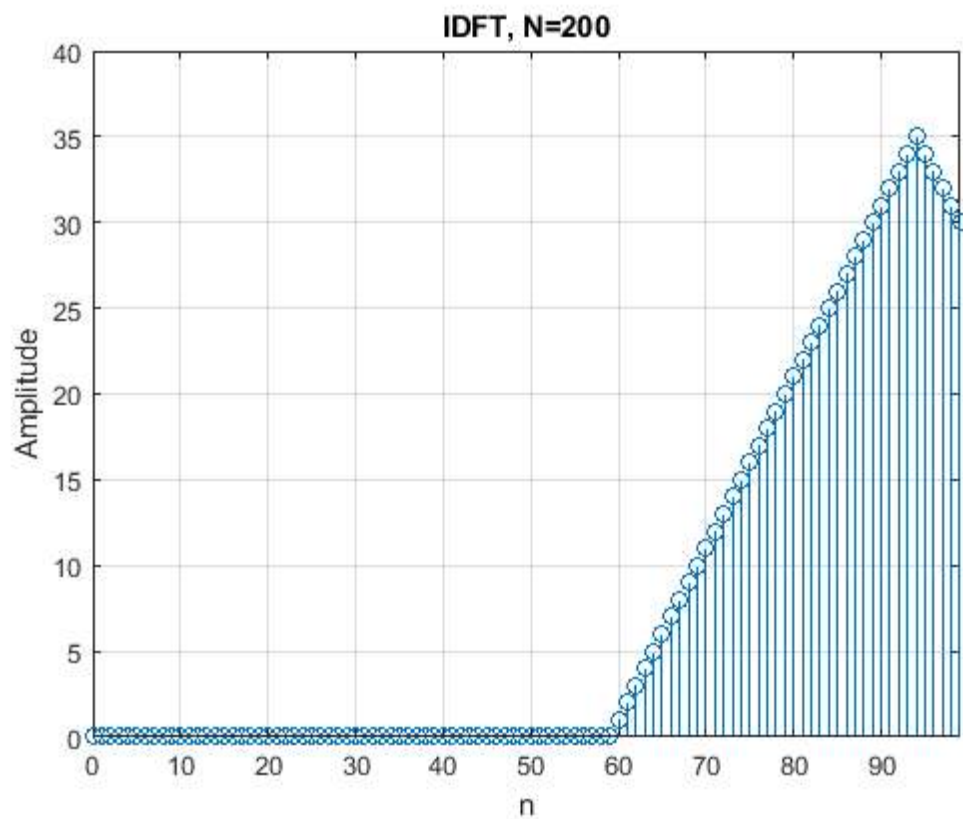
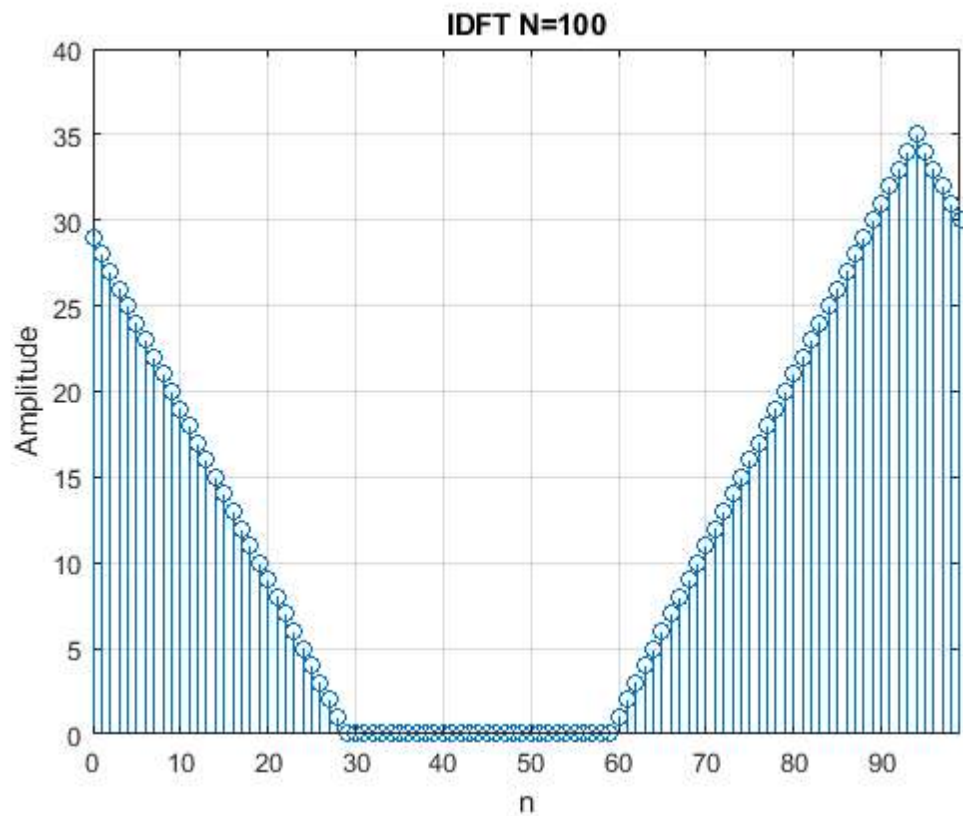
figure;
stem(0:99, y2_200(1:100));
xlabel('n');
ylabel('Amplitude');
title('IDFT, N=200');
xlim([0 99]);
grid on;

% -----
% 3(f)
% -----
% ANSWER QUESTION BELOW
% Because the length of x2's vector is from 0:99, it has 100 samples,
% which makes its required length  $2 \times 100 - 1 = 199$ . For  $N=100$ , it's less than
% the required amount which makes it wrap around itself. For  $N=200$ ,
%  $200 > 199$  so it meets the required threshold which makes it correct with
% no wrap around.

```

% =====





4. DFT AND FFT

Choose a song at least 3 minutes long to use in this problem, and include it in your submission. Load it into MATLAB using audioread. Note that most audio files will be stereo, so you need to make sure that you only use one column of audio data for this

```
[x3,fs] = audioread('Stateside + Zara Larsson - PinkPantheress .mp3');
x3 = x3(:,1);

% -----
% 4(a)
% -----
x3_10000 = x3(1:10000);

tic;
X3_DFT = DFT(x3_10000);
t_DFT = toc;
disp(['DFT computation time: ', num2str(t_DFT), ' seconds']);

f_axis = (0:9999) * fs / 10000;

figure;
plot(f_axis, abs(X3_DFT));
xlabel('Frequency (Hz)');
ylabel('Magnitude');
title('DFT Magnitude of First 10k Stateside Samples');
xlim([0 fs/2]);
grid on;

% -----
% 4(b)
% -----
tic;
X3_FFT = fft(x3_10000);
t_FFT = toc;
disp(['FFT computation time: ', num2str(t_FFT), ' seconds']);

f_axis = (0:9999) * fs / 10000;

figure;
plot(f_axis, abs(X3_FFT));
xlabel('Frequency (Hz)');
ylabel('Magnitude');
title('FFT Magnitude of First 10k Stateside Samples');
xlim([0 fs/2]);
grid on;

% -----
% 4(c)
% -----
% ANSWER QUESTION BELOW
% There is no noticeable different in the magnitude plots to me and this
% is because they are both different ways to compute the same thing. FFT is
% is the faster way to compute DFT.

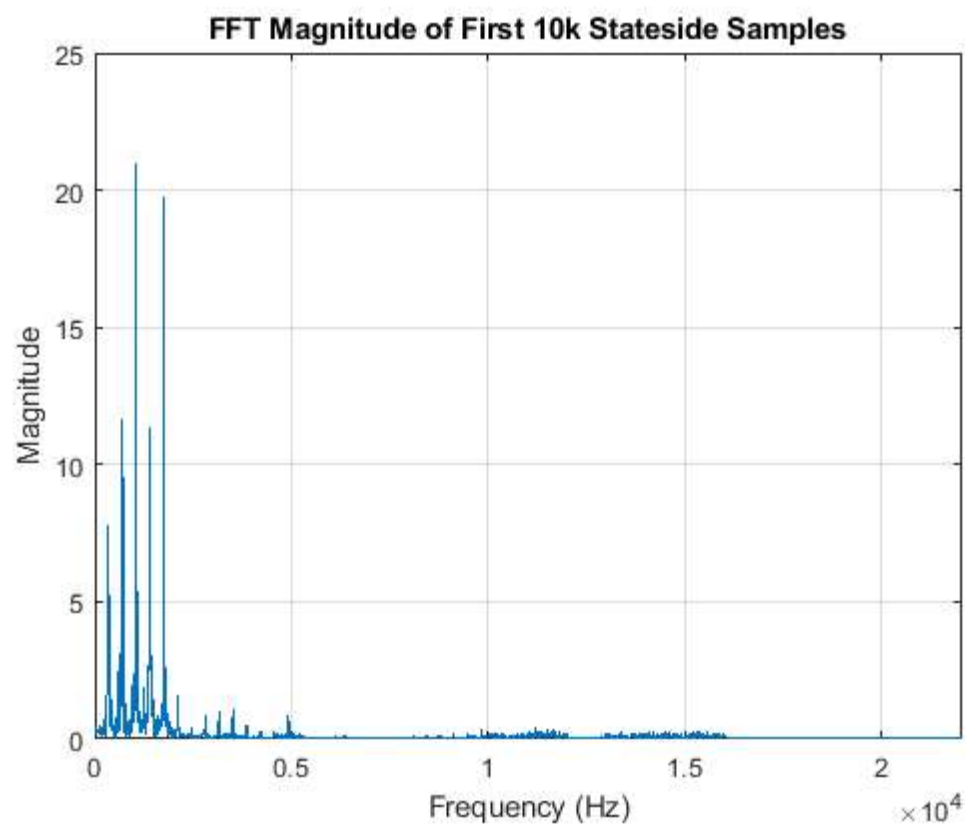
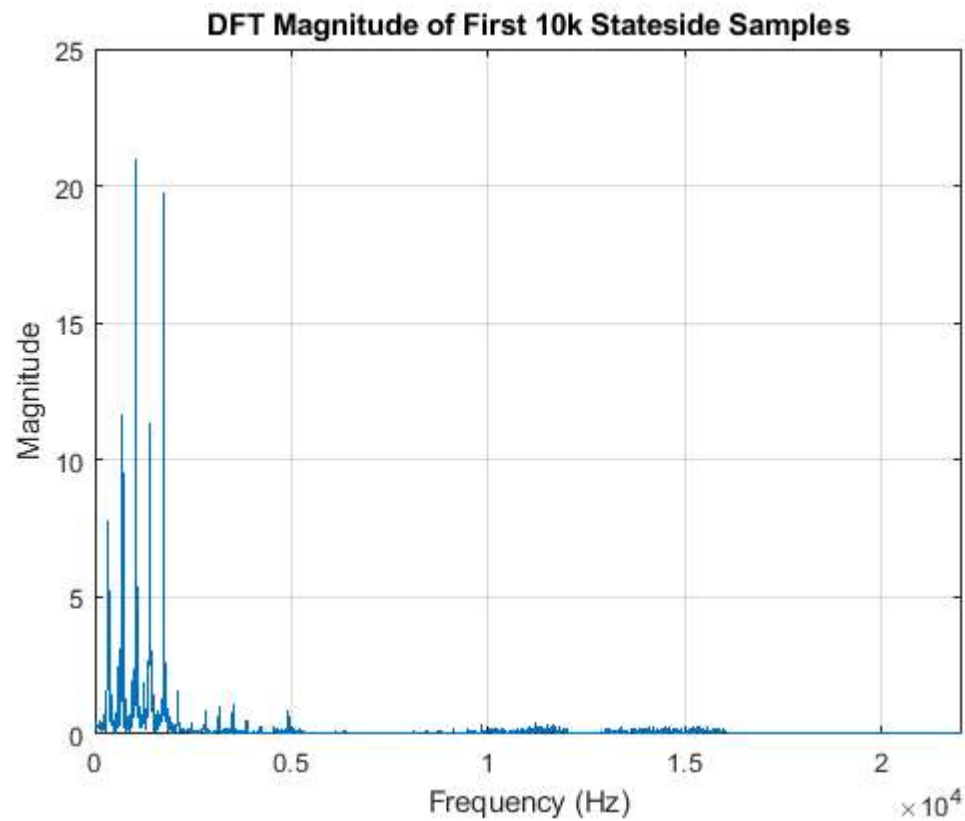
% -----
% 4(d)
```

```
% -----  
% ANSWER QUESTION BELOW  
% The FFT is 7,000 times faster than the DFT because my run says that DFT  
% computation time is 2.44 seconds and FFT computation time is 0.000345  
% seconds.  
%  
  
% -----  
% 4(e)  
% -----  
tic;  
X3_FFT_full = fft(x3);  
t_FFT_full = toc;  
disp(['FFT of entire song computation time: ', num2str(t_FFT_full), ' seconds']);
```

DFT computation time: 1.7948 seconds

FFT computation time: 0.0002325 seconds

FFT of entire song computation time: 0.14224 seconds



Functions provided for the lab

```
function H = DTFT(x,w)
% DTFT(X,W) compute the Discrete-time Fourier Transform of signal X
% across frequencies defined by W.
```

```

H = zeros(1, length(w));
for nn = 1:length(x)
    H = H + x(nn).*exp(-1j*w.*(nn-1));
end

end

function X = DFT(x)
% DFT(x) compute the N-point Discrete Fourier Transform of signal x
% Where N is the length of signal x
N = length(x);
w = (2*pi/N) * (0:N-1);
X = zeros(1, length(w));
for nn = 1:length(x)
    X = X + x(nn)*exp(-1j*w*(nn-1));
end

end

function x = IDFT(X)
% IDFT(x) compute the N-point Inverse Discrete Fourier Transform of signal
% X where N is the length of signal X
N = length(X); % FILL THIS LINE IN
w = (2*pi/N)*(0:N-1); % FILL THIS LINE IN
x = zeros(1, length(w));
for nn = 1:length(x)
    x = x + X(nn)*exp(1j*w*(nn-1)); % FILL THIS LINE IN
end

x = x/N;

end

```